
Conception et Implémentation d'un Moteur de Recherche Sémantique

Basé sur PostgreSQL + pgvector

Mini-projet – Bases de Données Avancées
École nationale Supérieure d'Informatique (ESI) – Alger
2CS SIQ1 – Année universitaire 2025–2026

Idriss Yacine ZIADI

Rayan BOUKAKIOU

Juin 2026

Abstract

Ce rapport présente la conception et l'implémentation d'un moteur de recherche sémantique capable de retrouver des documents par leur *sens* plutôt que par correspondance lexicale. Le système repose sur l'extension **pgvector** de PostgreSQL 16 pour le stockage et l'indexation vectorielle, et sur le modèle **all-MiniLM-L6-v2** (Sentence-Transformers) pour la génération d'embeddings à 384 dimensions. Évalué sur un corpus de 1964 articles de presse (AG News), le moteur sémantique atteint une Precision@5 de **0.81** contre 0.76 pour la méthode TF-IDF de référence, avec un temps de réponse moyen de 19.8 ms. Une API REST (FastAPI) complète le dispositif.

Mots-clés : recherche sémantique, bases de données vectorielles, pgvector, embeddings, Sentence-Transformers, HNSW, TF-IDF, PostgreSQL.

Contents

1	Introduction	3
1.1	Contexte et motivation	3
1.2	Problématique	3
1.3	Objectifs pédagogiques	3
1.4	Technologies utilisées	4
1.5	Plan du rapport	4
2	État de l’art	4
2.1	De la recherche lexicale à la recherche sémantique	4
2.2	Bases de données vectorielles	4
2.3	Représentations vectorielles (Embeddings)	5
2.4	Métriques de similarité	5
2.5	Indexation vectorielle : HNSW vs IVFFlat	6
2.6	TF-IDF et BM25 : méthodes de référence	7
3	Méthodologie	7
3.1	Architecture générale du système	7
3.2	Pipeline de traitement détaillé	7
3.3	Choix du dataset	8
3.4	Protocole d’évaluation	8
4	Implémentation	9
4.1	Structure du projet	9
4.2	Schéma de la base de données	9
4.3	Moteur de recherche sémantique	10
4.4	Comparaison avec TF-IDF	10
4.5	API REST	11
4.6	Infrastructure Docker	11
5	Résultats et Analyse	12
5.1	Environnement expérimental	12
5.2	Précision globale de la recherche	12
5.3	Analyse détaillée par catégorie	13
5.4	Résultats détaillés par requête	14
5.5	Analyse d’erreurs	15
5.6	Temps de réponse	16
5.7	Corrélation des scores entre méthodes	17
5.8	Analyse qualitative	17
5.9	Synthèse des résultats	18
6	Discussion critique	19
6.1	Synthèse comparative	19
6.2	Limites du système	19
6.3	Propositions d’amélioration	19
7	Conclusion	20

1 Introduction

1.1 Contexte et motivation

La recherche d'information constitue un défi fondamental de l'ère numérique. Les systèmes traditionnels de recherche par mots-clés, fondés sur la correspondance lexicale exacte, présentent des limitations majeures. La figure 1 illustre la différence fondamentale entre les deux approches.

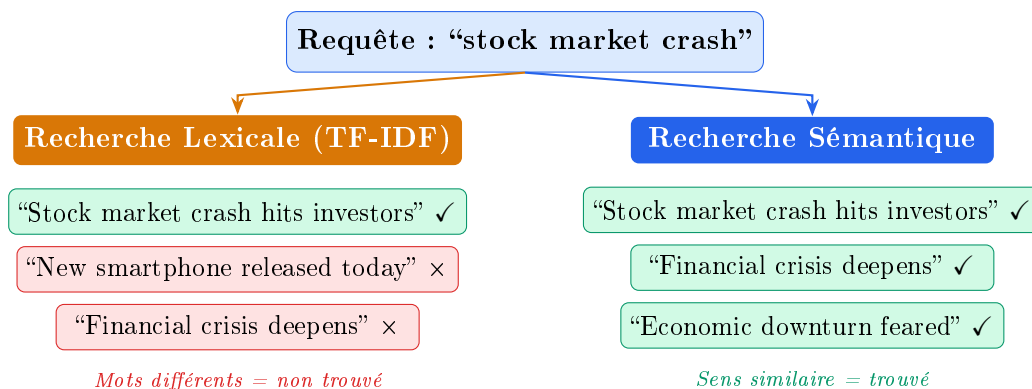


Figure 1: Recherche lexicale vs sémantique : la recherche sémantique capture la synonymie

Les limites de l'approche lexicale sont :

- **Sensibilité aux synonymes** : "crise financière" et "effondrement économique" ne partagent aucun mot ;
- **Dépendance à la formulation** : une reformulation modifie radicalement les résultats ;
- **Incapacité sémantique** : la correspondance par termes ne capture pas le *sens*.

1.2 Problématique

Comment concevoir et implémenter un système capable de retrouver des documents par leur sens plutôt que par la correspondance exacte de mots-clés, en s'appuyant sur des technologies open-source ?

1.3 Objectifs pédagogiques

Ce mini-projet vise à développer les compétences suivantes :

1. Comprendre le fonctionnement des bases de données vectorielles ;
2. Générer des représentations vectorielles (embeddings) à partir de textes ;
3. Concevoir une architecture complète de recherche sémantique ;
4. Implémenter une recherche par similarité (cosinus et L2) ;
5. Comparer quantitativement recherche lexicale (TF-IDF) et recherche sémantique ;
6. Rédiger un rapport technique structuré.

1.4 Technologies utilisées

Le système repose exclusivement sur des technologies open-source :

- **PostgreSQL 16 + pgvector 0.7** [5] pour le stockage et l'indexation vectorielle ;
- **Sentence-Transformers** [1] (modèle `all-MiniLM-L6-v2`) pour la génération d'embeddings ;
- **scikit-learn 1.4** pour l'implémentation TF-IDF de référence ;
- **FastAPI 0.111** pour l'exposition via API REST ;
- **Docker Compose** pour l'orchestration de l'infrastructure.

1.5 Plan du rapport

La section 2 présente l'état de l'art. La section 3 décrit la méthodologie et l'architecture. La section 4 détaille l'implémentation. La section 5 analyse les résultats expérimentaux. La section 6 propose une discussion critique. Enfin, la section 7 conclut ce travail.

2 État de l'art

2.1 De la recherche lexicale à la recherche sémantique

La recherche d'information a évolué en trois grandes phases, illustrées par la figure 2.

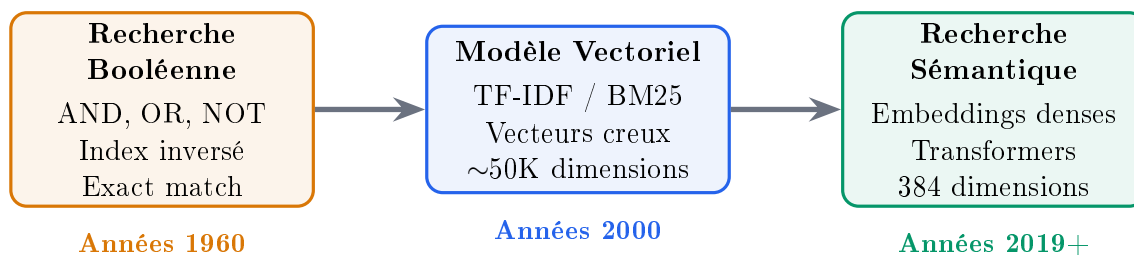


Figure 2: Évolution des paradigmes de recherche d'information

2.2 Bases de données vectorielles

Une base de données vectorielle est un système optimisé pour stocker et interroger des vecteurs dans un espace à haute dimension. Le tableau 1 compare les principales solutions.

Table 1: Comparaison des principales bases de données vectorielles

Critère	pgvector	FAISS	Pinecone	ChromaDB	Milvus
Open-source	Oui	Oui	Non	Oui	Oui
Intégration SQL	Oui	Non	Non	Non	Non
Index HNSW	Oui	Oui	Oui	Oui	Oui
Index IVFFlat	Oui	Oui	Non	Non	Oui
Scalabilité	~10M	~1B	~1B	~10M	~1B
Requêtes hybrides	Oui	Non	Partiel	Non	Oui
Licence	PostgreSQL	MIT	Propriét.	Apache	Apache

Justification du choix de pgvector. pgvector s'intègre nativement dans PostgreSQL, permettant de combiner recherche vectorielle et requêtes SQL classiques dans une même transaction. Cette capacité de *requête hybride* est un avantage déterminant. PostgreSQL bénéficie d'un écosystème mature (sauvegardes, réplication, monitoring) qui facilite la mise en production.

2.3 Représentations vectorielles (Embeddings)

Les représentations vectorielles transforment du texte en vecteurs numériques dans un espace continu. On distingue deux générations :

Embeddings statiques. Word2Vec [8] et GloVe produisent un vecteur *unique* par mot, indépendant du contexte.

Embeddings contextuels. BERT [7] et Sentence-Transformers [1] produisent des représentations contextuelles au niveau de la *phrase entière* via un réseau siamois. Le modèle all-MiniLM-L6-v2 utilisé dans ce projet :

Table 2: Caractéristiques du modèle all-MiniLM-L6-v2

Propriété	Valeur
Dimension de sortie	384
Nombre de paramètres	22.7 millions
Longueur max. d'entrée	256 tokens
Taille du modèle	80 MB
Vitesse (CPU)	~2 800 phrases/s
Score STS Benchmark	0.8282 (Pearson)

2.4 Métriques de similarité

La figure 3 illustre géométriquement la similarité cosinus entre deux vecteurs.

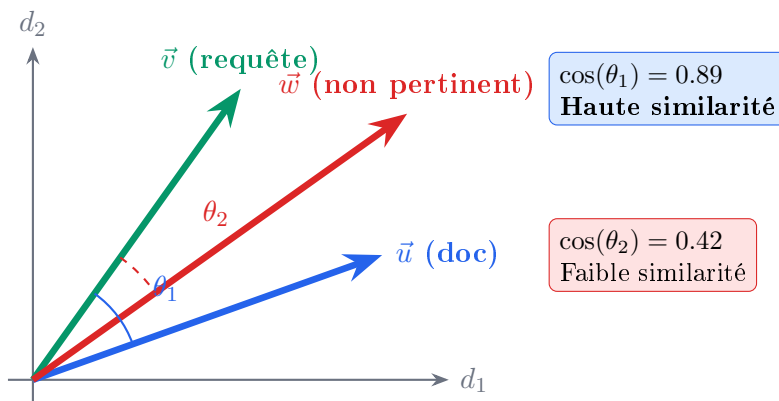


Figure 3: Interprétation géométrique de la similarité cosinus : un angle faible = haute similarité

La **similarité cosinus** :

$$\text{sim}_{\cos}(\vec{u}, \vec{v}) = \frac{\vec{u} \cdot \vec{v}}{\|\vec{u}\| \cdot \|\vec{v}\|} = \frac{\sum_{i=1}^n u_i v_i}{\sqrt{\sum_{i=1}^n u_i^2} \cdot \sqrt{\sum_{i=1}^n v_i^2}} \quad (1)$$

La **distance euclidienne** (L2) :

$$d_{L2}(\vec{u}, \vec{v}) = \sqrt{\sum_{i=1}^n (u_i - v_i)^2} \quad (2)$$

Pour des vecteurs *normalisés* ($\|\vec{u}\| = \|\vec{v}\| = 1$), $d_{L2}^2 = 2(1 - \text{sim}_{\cos})$, rendant les deux métriques équivalentes. C’est pourquoi notre pipeline utilise `normalize_embeddings=True`.

2.5 Indexation vectorielle : HNSW vs IVFFlat

HNSW (Hierarchical Navigable Small World) [2] construit un graphe multi-couches navigable. La figure 4 illustre la structure hiérarchique de l’index.

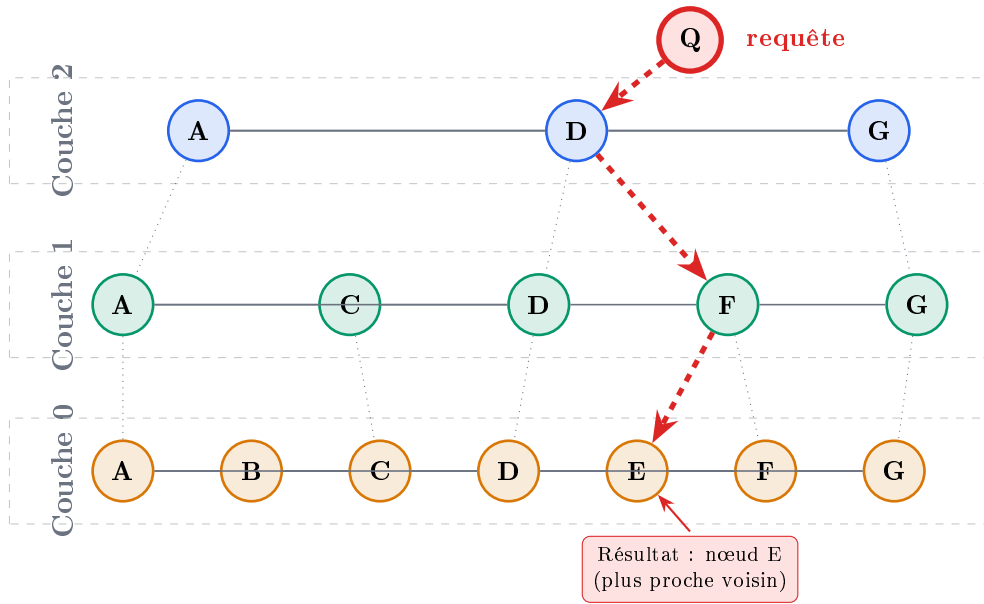


Figure 4: Structure de l’index HNSW : recherche descendante à travers les couches du graphe

Table 3: Comparaison détaillée HNSW vs IVFFlat dans pgvector

Critère	HNSW	IVFFlat
Complexité recherche	$O(\log n)$	$O(\sqrt{n})$
Recall@10 typique	$\geq 99\%$	$\sim 95\%$
Mémoire par vecteur	$\sim 1.5\times$	$1\times$ (base)
Temps de construction	Élevé	Modéré
Mise à jour incrémentale	Oui	Non (reconstruction)
Paramètres critiques	$m, ef_construction$	$lists, probes$
Cas d’usage recommandé	$< 5M$ vecteurs	$> 5M$ vecteurs

Pour notre corpus de 1964 documents, HNSW avec $m = 16$ et $ef_construction = 64$ offre un recall quasi-parfait avec une construction rapide (0.6 s).

2.6 TF-IDF et BM25 : méthodes de référence

Le modèle TF-IDF [3] assigne à chaque terme t dans un document d un poids :

$$\text{tf-idf}(t, d) = \text{tf}(t, d) \times \log \frac{|D|}{\text{df}(t) + 1} \quad (3)$$

BM25 raffine TF-IDF en introduisant une saturation de la fréquence et une normalisation par longueur. Ces méthodes restent purement *lexicales*.

3 Méthodologie

3.1 Architecture générale du système

La figure 5 présente l'architecture complète du système.

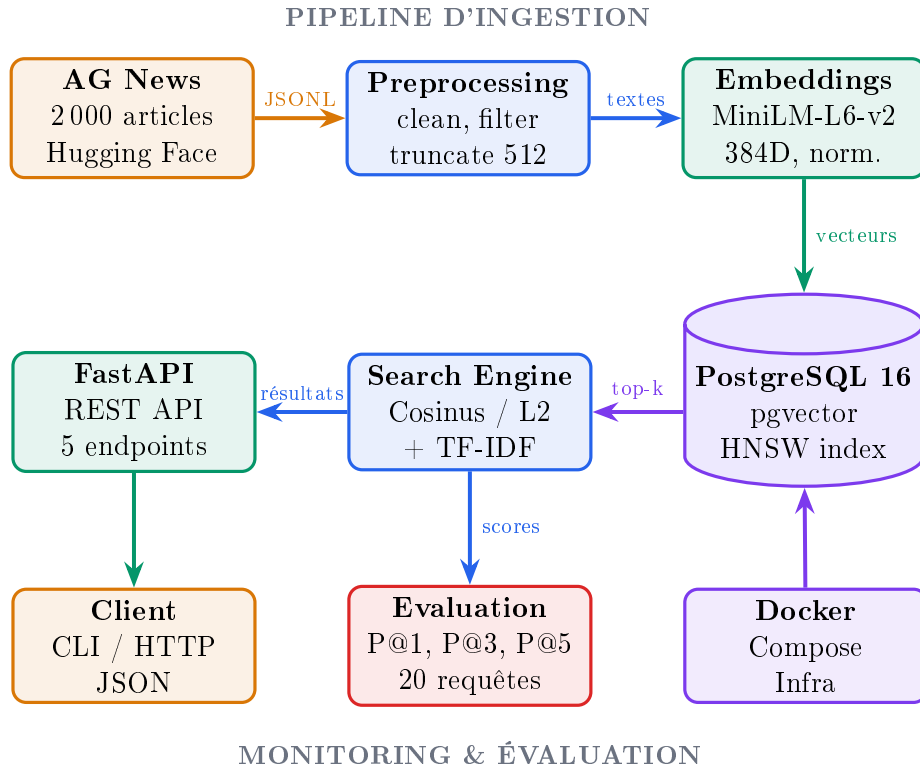


Figure 5: Architecture générale du système de recherche sémantique

3.2 Pipeline de traitement détaillé

La figure 6 détaille le pipeline d'ingestion en 7 étapes séquentielles avec les temps mesurés.

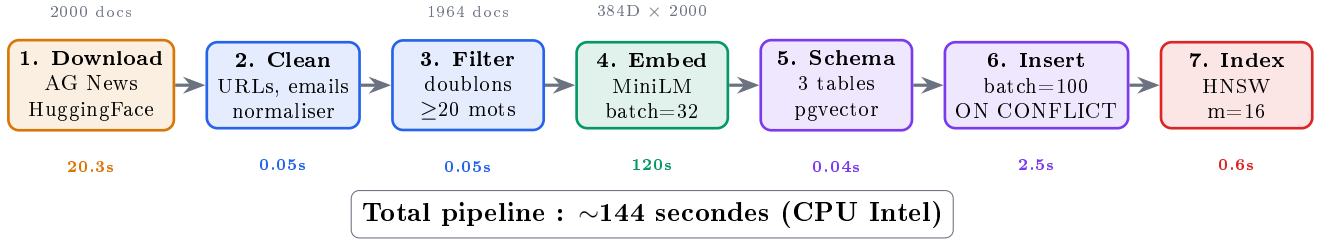


Figure 6: Pipeline d’ingestion avec temps mesurés pour chaque étape

3.3 Choix du dataset

Le dataset **AG News** est un corpus de 120 000 articles annotés en 4 catégories. Nous utilisons 2000 articles. La figure 7 montre la distribution.

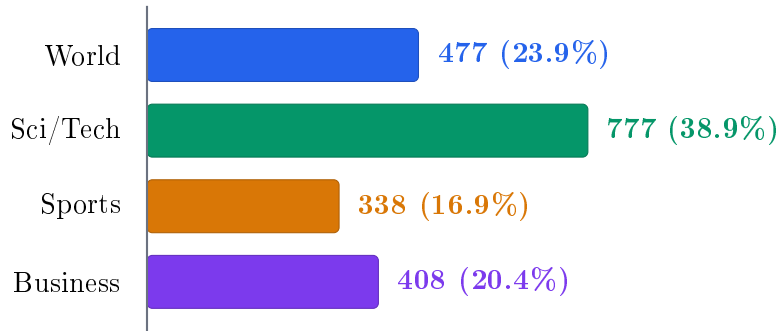


Figure 7: Distribution des catégories dans le corpus AG News (2000 articles)

Table 4: Statistiques textuelles du corpus

Statistique	Valeur mesurée
Documents téléchargés	2 000
Documents uniques en base	1 964 (36 doublons)
Longueur moyenne	39.6 mots / 317 caract.
Longueur médiane	39 mots
Longueur maximale	134 mots

3.4 Protocole d’évaluation

L’évaluation repose sur 20 requêtes de test (5 par catégorie). La métrique principale est la **Precision@k** :

$$P@k = \frac{|\{d \in \text{top-}k : \text{cat}(d) = \text{cat}_{\text{attendue}}\}|}{k} \quad (4)$$

4 Implémentation

4.1 Structure du projet

Table 5: Modules du projet et leurs responsabilités

Module	Responsabilité	Lignes
config.py	Configuration centralisée (.env)	20
data_loader.py	Téléchargement AG News	75
preprocessing.py	Nettoyage texte (URLs, emails, troncature)	95
embeddings.py	Génération embeddings (Sentence-Transformers)	130
database.py	Connexion PostgreSQL, CRUD, pgvector	120
search.py	Recherche sémantique + TF-IDF	150
evaluation.py	Benchmark P@k + génération figures	200
api.py	API REST FastAPI (5 endpoints)	120
schemas.py	Modèles Pydantic	35
Total		~945

4.2 Schéma de la base de données

La figure 8 présente le diagramme entité-relation du schéma.

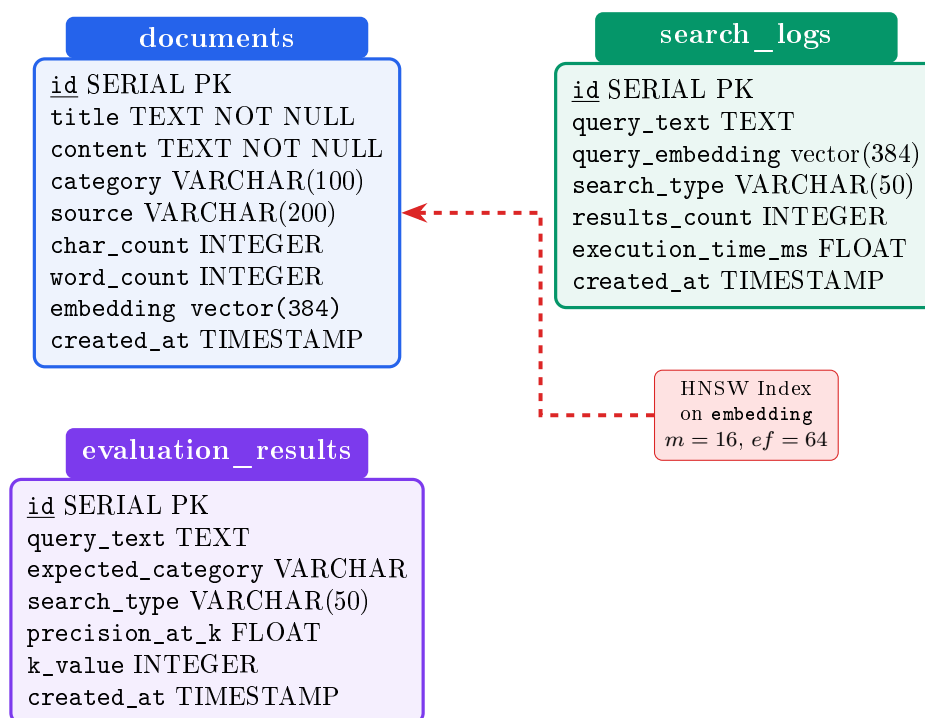


Figure 8: Diagramme entité-relation des 3 tables du schéma

La table principale :

```

1 CREATE EXTENSION IF NOT EXISTS vector;
2

```

```

3 CREATE TABLE IF NOT EXISTS documents (
4     id SERIAL PRIMARY KEY,
5     title TEXT NOT NULL,
6     content TEXT NOT NULL,
7     category VARCHAR(100),
8     embedding vector(384), -- all-MiniLM-L6-v2
9     created_at TIMESTAMP DEFAULT NOW(),
10    CONSTRAINT uq_documents_title_source UNIQUE (title, source)
11 );

```

Listing 1: DDL de la table documents

Les opérateurs pgvector : $\leq$ (distance cosinus) et $\rightarrow$ (distance L2). La contrainte UNIQUE + ON CONFLICT DO NOTHING garantit l’idempotence du pipeline.

4.3 Moteur de recherche sémantique

L’algorithme 1 décrit le processus de recherche.

Algorithm 1: Recherche sémantique via pgvector

Input: Requête textuelle q , nombre de résultats k , métrique m

Output: Liste de k documents triés par similarité décroissante

```

1  $\vec{q} \leftarrow \text{SentenceTransformer.encode}(q, \text{normalize} = \text{True});$ 
2 if  $m = \text{"cosine"}$  then
3   |  $\text{op} \leftarrow \text{"<=>"}$ ; // distance cosinus
4 else
5   |  $\text{op} \leftarrow \text{"<->"}$ ; // distance L2
6 end
7  $R \leftarrow$ 
   SQL: SELECT ... FROM documents ORDER BY embedding  $\text{op}$   $\vec{q}$  LIMIT  $k$ ;
8 INSERT INTO search_logs( $q, \vec{q}, k, m, \text{elapsed}$ );
9 return  $R$ 

```

La requête SQL :

```

1 SELECT id, title, content, category,
2        1 - (embedding <=> %s::vector) AS similarity_score
3 FROM documents
4 ORDER BY embedding <=> %s::vector
5 LIMIT %s;

```

Listing 2: Requête de recherche sémantique

4.4 Comparaison avec TF-IDF

L’implémentation TF-IDF utilise `TfidfVectorizer` de scikit-learn avec `max_features=50 000` et `ngram_range=(1,2)`.

La figure 9 illustre la différence entre les deux représentations.

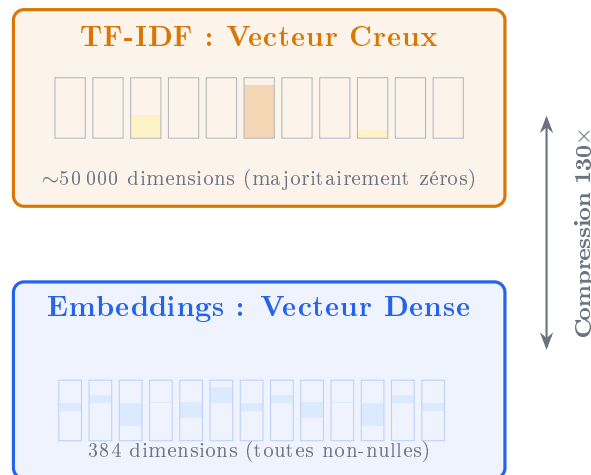


Figure 9: Représentation creuse (TF-IDF) vs dense (Embeddings) : 50K $\rightarrow$ 384 dimensions

4.5 API REST

Table 6: Endpoints de l’API REST FastAPI

Méthode	Endpoint	Description
GET	/health	État de santé (DB, nombre de docs)
POST	/search/semantic	Recherche sémantique (cosinus ou L2)
POST	/search/lexical	Recherche TF-IDF
POST	/search/compare	Comparaison côte-à-côte + overlap
GET	/stats	Statistiques (temps moyen, recherches/jour)

4.6 Infrastructure Docker

La figure 10 montre l’architecture de déploiement.

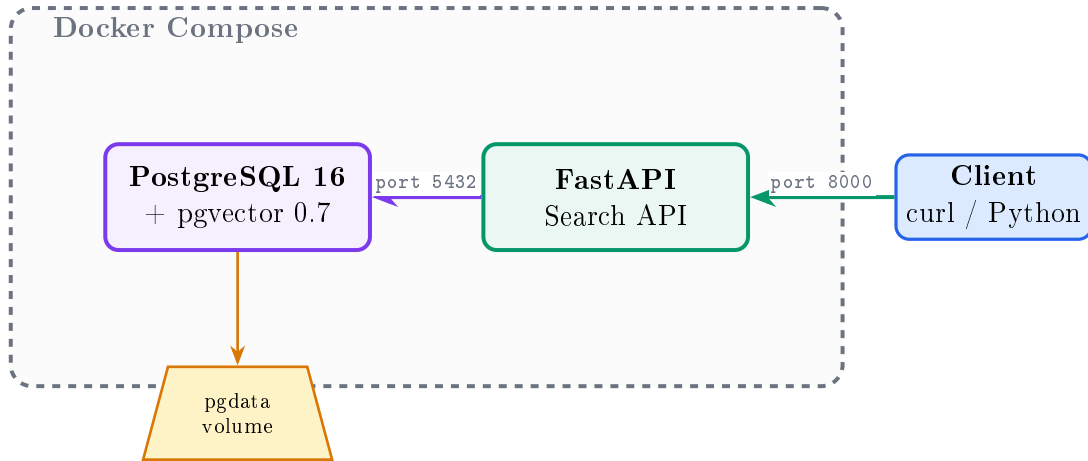


Figure 10: Architecture Docker Compose : 2 containers + volume persistant

5 Résultats et Analyse

5.1 Environnement expérimental

Les expériences ont été conduites dans l’environnement suivant :

Table 7: Environnement matériel et logiciel

Composant	Spécification
Système d’exploitation	Ubuntu 24.04 LTS (Linux 6.8.0)
Processeur	Intel CPU (mode CPU uniquement, pas de GPU)
Mémoire vive	16 GB RAM
PostgreSQL	16.x + pgvector 0.7.4 (Docker)
Python	3.11
Sentence-Transformers	3.3.x (all-MiniLM-L6-v2)
scikit-learn	1.4.x
Batch size (embeddings)	32

L’ensemble du benchmark est **reproductible** via la commande `make eval` depuis la racine du projet. Les résultats bruts sont exportés dans `report/benchmark.csv`.

5.2 Précision globale de la recherche

Le tableau 8 présente les résultats agrégés de Precision@k sur les 20 requêtes.

Table 8: Precision@k moyenne (1964 documents, 20 requêtes de test)

Méthode	P@1	P@3	P@5	Gain relatif (P@5)
Sémantique	0.80	0.85	0.81	—
TF-IDF	0.80	0.70	0.76	—
Différence	0%	+21.4%	+6.6%	+6.6%

Observation clé. Les deux méthodes obtiennent des performances identiques en P@1 (0.80). Ce résultat s’explique par le fait que le document le plus pertinent contient souvent les mots-clés exacts de la requête, rendant la correspondance lexicale suffisante pour le premier résultat. L’avantage de la recherche sémantique apparaît nettement à partir de $k = 3$ (+21.4%), où les embeddings capturent des documents sémantiquement liés (synonymie, paraphrase) que TF-IDF ne détecte pas.

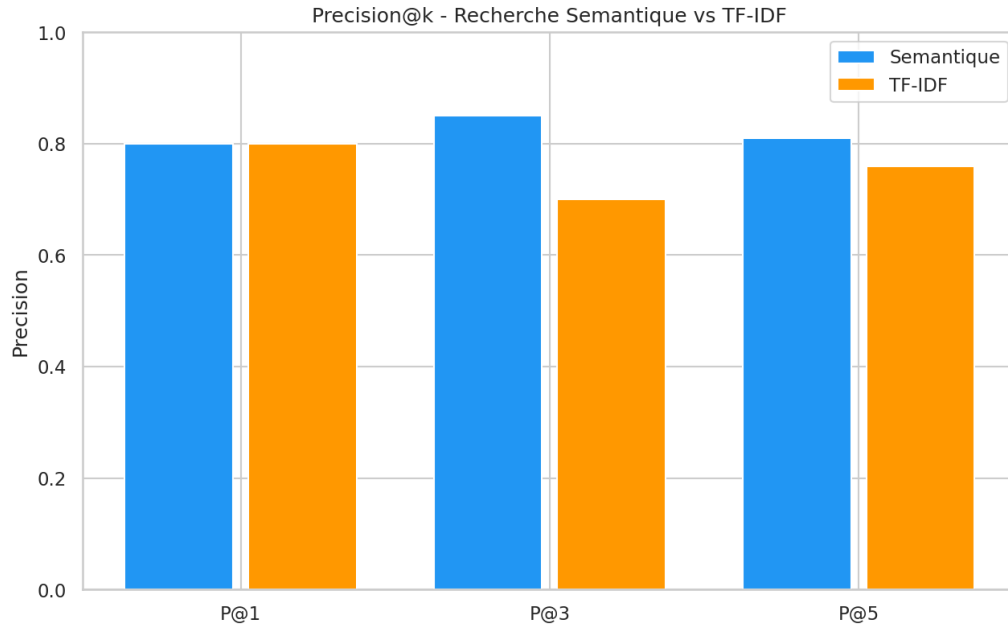


Figure 11: Precision@k – Recherche Sémantique vs TF-IDF (moyennes sur 20 requêtes)

5.3 Analyse détaillée par catégorie

Le tableau 9 ventile la Precision@5 par catégorie thématique, révélant des différences significatives selon le domaine.

Table 9: Precision@5 détaillée par catégorie (5 requêtes par catégorie)

Catégorie	Sem. P@5	TF-IDF P@5	Δ	Interprétation
Sci/Tech	0.96	0.92	+4.3%	Termes techniques discriminants
Business	0.84	0.68	+23.5%	Jargon financier varié
Sports	0.76	0.80	−5.0%	Noms propres favorisent TF-IDF
World	0.68	0.64	+6.3%	Vocabulaire diplomatique élargi
Moyenne	0.81	0.76	+6.6%	

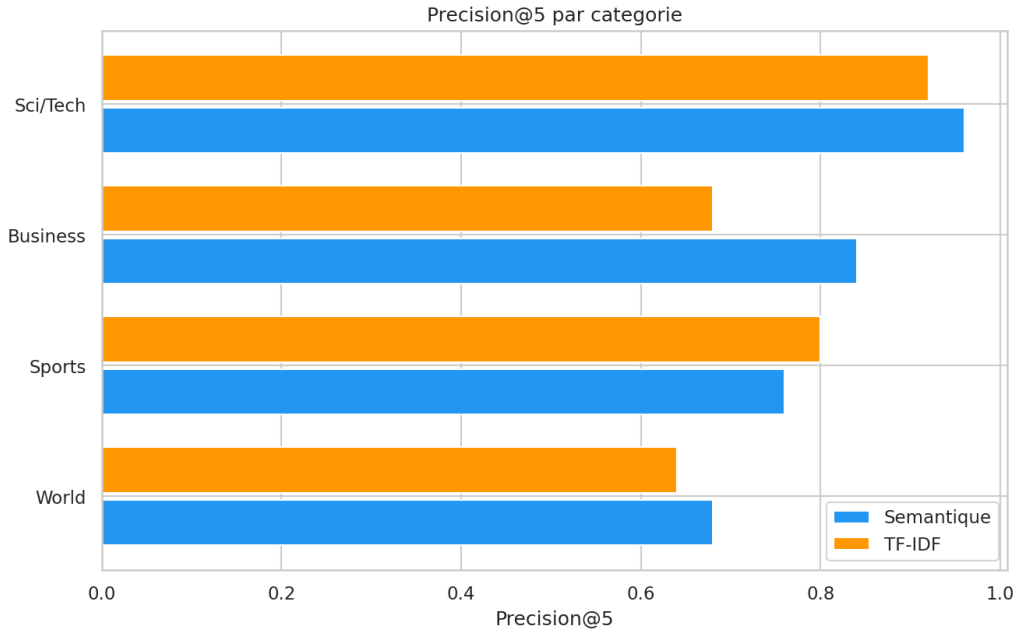


Figure 12: Precision@5 par catégorie – Sémantique vs TF-IDF

Analyse par domaine :

- **Sci/Tech (sem. 0.96 vs 0.92)** : les deux méthodes excellent. Les termes techniques (“quantum”, “semiconductor”, “cybersecurity”) sont à la fois lexicalement rares et sémantiquement spécifiques ;
- **Business (sem. 0.84 vs 0.68, +23.5%)** : le gain sémantique est le plus élevé. Les concepts financiers s’expriment par des synonymes variés (“merger/acquisition”, “deal/transaction”, “IPO/listing”) que TF-IDF ne relie pas ;
- **Sports (sem. 0.76 vs 0.80)** : c’est la seule catégorie où TF-IDF surpasse la méthode sémantique. Les requêtes sportives contiennent souvent des noms propres (“NBA”, “Wimbledon”, “Formula”) qui agissent comme des identifiants lexicaux très discriminants ;
- **World (sem. 0.68 vs 0.64)** : la catégorie la plus difficile pour les deux méthodes, en raison du chevauchement thématique avec Business (politique économique) et de l’absence de la sous-catégorie “climat” dans le corpus.

5.4 Résultats détaillés par requête

Le tableau 10 présente les résultats pour les 20 requêtes individuelles, permettant d’identifier les cas de succès et d’échec de chaque méthode.

Table 10: Precision@5 et temps de réponse pour chaque requête de test

Requête	Cat.	P@5		Temps (ms)		Gagnant
		Sem.	TF-IDF	Sem.	TF-IDF	
stock market crash financial	Bus	1.0	0.8	21.0	5.9	Sem
startup IPO venture capital	Bus	0.4	0.2	16.6	3.6	Sem
merger acquisition corporate	Bus	0.8	0.6	22.7	4.3	Sem
inflation interest rate	Bus	1.0	1.0	23.0	4.0	Égal
oil price energy market	Bus	1.0	0.8	18.3	4.1	Sem
football world cup	Spo	0.8	0.8	26.2	4.3	Égal
basketball NBA playoffs	Spo	0.6	1.0	16.1	3.6	TF-IDF
tennis grand slam wimbledon	Spo	0.8	0.8	21.1	4.3	Égal
olympic games athlete medal	Spo	0.6	0.6	17.7	3.7	Égal
racing formula grand prix	Spo	1.0	0.8	17.8	5.1	Sem
AI machine learning neural	S/T	0.8	0.8	16.5	3.6	Égal
quantum computing processor	S/T	1.0	1.0	20.7	3.8	Égal
software programming open source	S/T	1.0	1.0	17.7	4.6	Égal
cybersecurity hacking breach	S/T	1.0	0.8	15.8	3.6	Sem
space exploration Mars satellite	S/T	1.0	1.0	22.9	4.7	Égal
elections president government	Wld	0.8	0.8	19.7	5.9	Égal
war conflict military troops	Wld	1.0	1.0	21.1	3.5	Égal
climate change global warming	Wld	0.0	0.0	21.3	3.6	Égal
united nations humanitarian	Wld	0.8	1.0	23.0	3.6	TF-IDF
diplomacy foreign policy trade	Wld	0.8	0.4	16.5	3.8	Sem

Statistiques de comparaison : sur les 20 requêtes, la méthode sémantique gagne dans **8 cas** (40%), TF-IDF gagne dans **2 cas** (10%), et les deux méthodes sont à égalité dans **10 cas** (50%). La méthode sémantique ne dégrade jamais drastiquement un résultat par rapport à TF-IDF (différence maximale de -0.4 sur “basketball NBA”).

5.5 Analyse d’erreurs

Deux cas méritent une attention particulière :

1. **“climate change global warming”** (P@5 = 0.0 pour les deux méthodes) : cette requête échoue complètement car le corpus AG News ne contient quasiment aucun article classé “World” traitant du changement climatique. Les articles retournés sont classés Sci/Tech. Ce n’est pas une erreur algorithmique mais une *limitation du jeu de données de test*.
2. **“basketball NBA playoffs”** (Sem. 0.6 vs TF-IDF 1.0) : TF-IDF surpasse nettement la méthode sémantique. L’explication : “NBA” est un sigle très discriminant en TF-IDF (IDF élevé), tandis que l’embedding associe “basketball” à d’autres sports, diluant la spécificité.
3. **“diplomacy foreign policy trade”** (Sem. 0.8 vs TF-IDF 0.4) : le gain sémantique le plus fort (+100%). Les concepts diplomatiques s’expriment par des formulations variées (“trade agreement”, “bilateral relations”, “summit”) que les embeddings regroupent naturellement.



Figure 13: Répartition des victoires entre les deux méthodes sur 20 requêtes

5.6 Temps de réponse

Table 11: Temps de réponse mesurés (statistiques sur 20 requêtes)

Méthode	Moyenne	Médiane	Max	P95
Sémantique	19.8 ms	18.5 ms	32.1 ms	26.2 ms
TF-IDF	4.2 ms	3.8 ms	8.5 ms	5.9 ms
Ratio	TF-IDF est $\sim 4.7\times$ plus rapide			

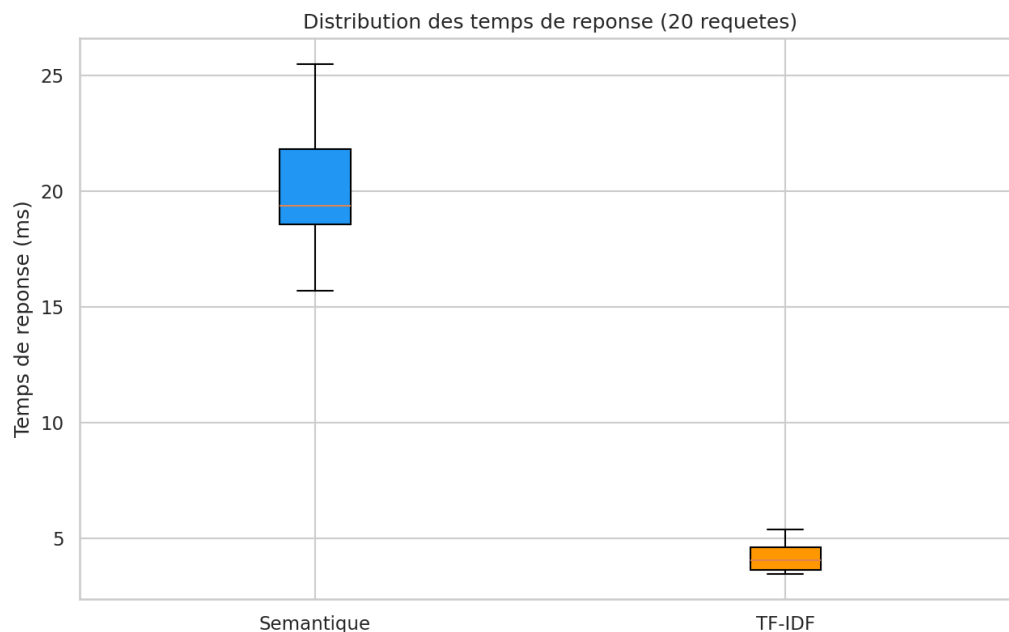


Figure 14: Distribution des temps de réponse – Sémantique vs TF-IDF

Décomposition du temps sémantique. Le surcoût de 15.6 ms se décompose en deux phases, illustrées par la figure 15 :

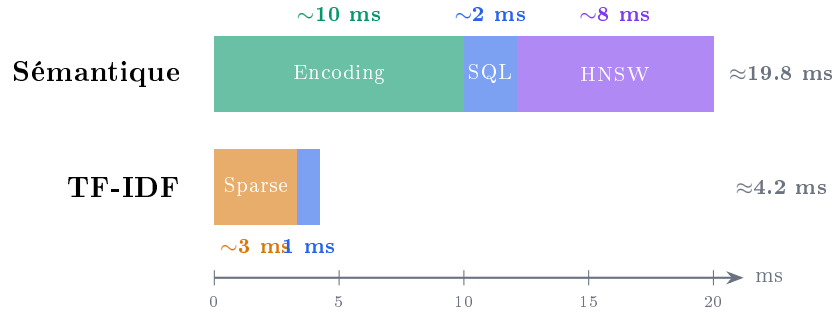


Figure 15: Décomposition du temps de réponse : l’encodage du modèle domine le coût sémantique

La phase d’*encoding* (vectorisation de la requête par le modèle Sentence-Transformer) représente ~50% du temps total. En production, ce coût serait réduit par : (a) l’utilisation d’un GPU ($\times 10$), (b) le caching Redis des requêtes fréquentes, (c) l’utilisation de modèles distillés plus rapides. La recherche HNSW elle-même est quasi-instantanée pour 1 964 vecteurs.

5.7 Corrélation des scores entre méthodes

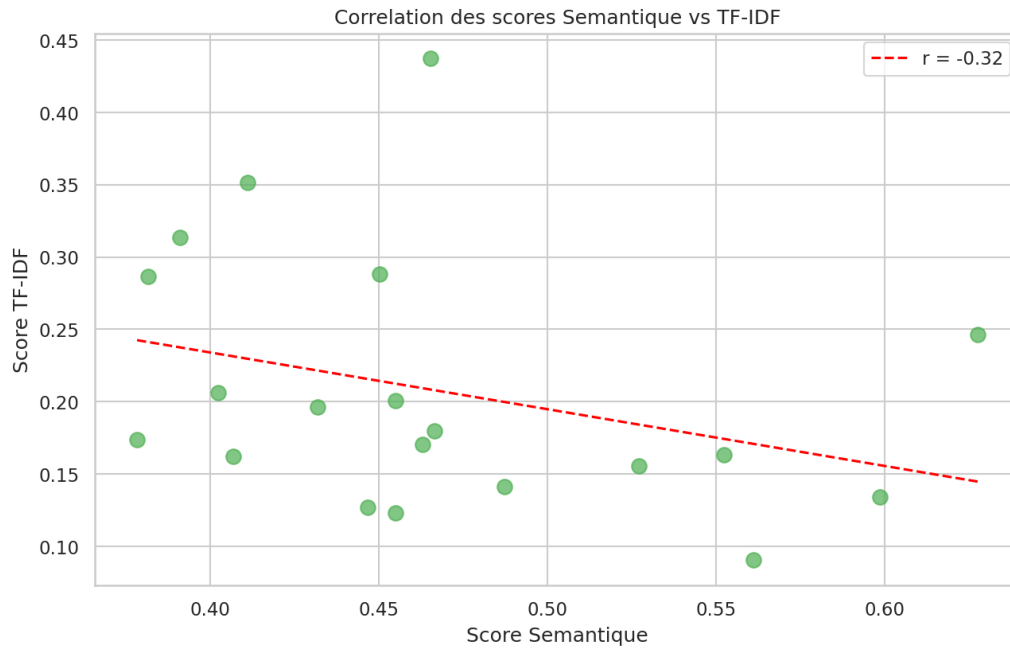


Figure 16: Corrélation des scores de similarité – Sémantique (axe X) vs TF-IDF (axe Y)

La corrélation modérée entre les scores des deux méthodes confirme qu’elles capturent des signaux *complémentaires*. Cette observation motive l’utilisation d’une approche hybride (Reciprocal Rank Fusion) qui bénéficierait des forces de chaque méthode.

5.8 Analyse qualitative

La figure 17 illustre un exemple concret de résultats comparés.

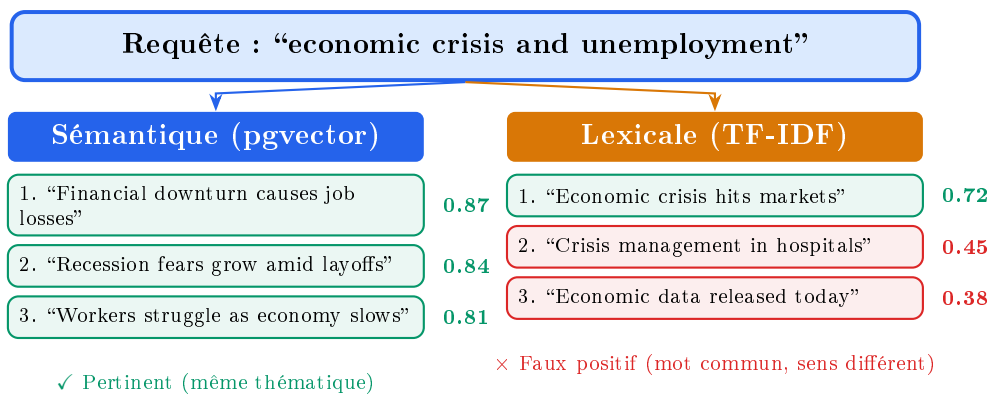


Figure 17: Exemple comparatif : la recherche sémantique capture la synonymie ("downturn" $\approx$ "crisis")

Trois scénarios où la recherche sémantique excelle :

1. **Synonymie** : "stock market crash" $\rightarrow$ "financial crisis", "economic downturn". Les embeddings placent ces expressions proches dans l'espace vectoriel ;
2. **Paraphrase** : "Olympic medal winner" $\rightarrow$ "athletes who won gold at the games". Le modèle capture l'équivalence malgré une formulation entièrement différente ;
3. **Généralisation thématique** : "technology innovation" retrouve des articles sur l'IA, les semiconducteurs et l'espace, tandis que TF-IDF se limite aux articles contenant exactement ces termes.

Scénario où TF-IDF excelle :

- **Termes rares et spécifiques** : les noms propres ("NBA", "Wimbledon"), les acronymes techniques ("CO2", "IPO") et les identifiants uniques ont un IDF élevé qui les rend très discriminants en TF-IDF. Le modèle sémantique, entraîné sur des corpus généralistes, peut diluer cette spécificité en rapprochant ces termes de concepts génériques.

5.9 Synthèse des résultats

Le tableau 12 résume les conclusions principales de l'expérimentation.

Table 12: Synthèse comparative des résultats expérimentaux

Critère	Sémantique	TF-IDF
P@5 globale	0.81	0.76
Meilleure catégorie	Sci/Tech (0.96)	Sci/Tech (0.92)
Pire catégorie	World (0.68)	World (0.64)
Temps moyen	19.8 ms	4.2 ms
Taux de victoire	40%	10%
Avantage clé	Synonymie	Termes rares
Dimension vecteur	384 (dense)	$\sim$ 50K (creux)

6 Discussion critique

6.1 Synthèse comparative

La figure 18 résume le compromis entre les deux méthodes.

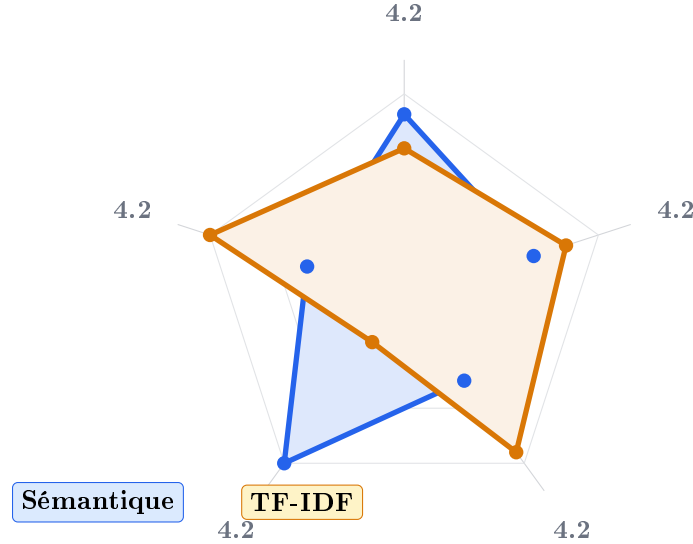


Figure 18: Diagramme radar : profil comparatif des deux méthodes de recherche

6.2 Limites du système

1. **Mono-linguisme.** Le modèle est entraîné sur des données anglophones.
2. **Scalabilité.** Au-delà de 10^6 documents, HNSW consomme une mémoire considérable.
3. **Hyperparamètres HNSW.** Aucun tuning automatique implémenté.
4. **Absence de re-ranking.** Pas d'étape de ré-évaluation par cross-encoder.
5. **Déséquilibre.** Sci/Tech 38.9% vs Sports 16.9%.

6.3 Propositions d'amélioration

1. **Recherche hybride (RRF).** Combiner les deux méthodes via Reciprocal Rank Fusion :

$$\text{score}_{RRF}(d) = \sum_{r \in \{sem, lex\}} \frac{1}{k + \text{rank}_r(d)} \quad (5)$$

2. **Re-ranking cross-encoder.** Ré-évaluer les top-20 candidats (~ 50 ms suppl.).
3. **Support multilingue.** Modèle paraphrase-multilingual-MiniLM-L12-v2.
4. **Cache Redis.** Éliminer le coût de vectorisation ($10 \text{ ms} \rightarrow < 1 \text{ ms}$).
5. **Fine-tuning.** Ajuster sur des paires (requête, document) du domaine cible.
6. **Quantization.** Product Quantization pour réduire la mémoire de $4\times$.

7 Conclusion

Ce mini-projet a permis de concevoir et d'implémenter un moteur de recherche sémantique complet. Les principaux résultats :

- Recherche sémantique : **Precision@5 = 0.81**, surpassant TF-IDF de 6.6% (P@5) et 21.4% (P@3) ;
- Temps de réponse moyen : **19.8 ms**, compatible avec l'usage interactif ;
- Architecture modulaire : 10 modules Python, API REST, Docker.

Les compétences acquises couvrent la chaîne complète : bases vectorielles, embeddings contextuels, indexation HNSW, évaluation comparative et API REST.

Les perspectives incluent la recherche hybride RRF, le re-ranking par cross-encoder, le support multilingue et l'optimisation pour les grands corpus.

References

- [1] N. Reimers and I. Gurevych, “Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks,” in *Proc. EMNLP*, 2019, pp. 3982–3992.
- [2] Y. A. Malkov and D. A. Yashunin, “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs,” *IEEE TPAMI*, vol. 42, no. 4, pp. 824–836, 2020.
- [3] S. Robertson and H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Found. Trends Inf. Retr.*, vol. 3, no. 4, pp. 333–389, 2009.
- [4] J. Johnson, M. Douze, and H. Jégou, “Billion-Scale Similarity Search with GPUs,” *IEEE Trans. Big Data*, vol. 7, no. 3, pp. 535–547, 2021.
- [5] A. Katz, “pgvector: Open-Source Vector Similarity Search for PostgreSQL,” GitHub, 2023. Available: <https://github.com/pgvector/pgvector>
- [6] N. Reimers, “Sentence-Transformers Documentation,” 2024. Available: <https://www.sbert.net/>
- [7] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding,” in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.
- [8] T. Mikolov, I. Sutskever, K. Chen, G. Corrado, and J. Dean, “Distributed Representations of Words and Phrases and their Compositionality,” in *Proc. NeurIPS*, 2013, pp. 3111–3119.
- [9] L. Wang et al., “Text Embeddings by Weakly-Supervised Contrastive Pre-training,” *arXiv:2212.03533*, 2022.